DATA QUALITY ASSURANCE | TECHNICAL CHALLENGE

Employee 360
Data Quality

From raw source data to a governed release decision

Profiling • reconciliation • monitoring • Bronze / Silver / Gold
Azure Databricks evidence • operational observability • data governance

RELEASE DECISION

BLOCK

The controls found source-data defects and prevent incorrect records reaching Gold.

Prepared by

Anthony Grant Apollis

October 2026 • South Africa

Final validation note

Anthony Grant Apollis | 11 October 2026

Read this note with the historical Databricks evidence

The original eBook pages describe earlier cloud runs. Their figures and screenshots are retained as historical evidence. They do not certify that the later local Python, SQL, PySpark and dbt files are identical implementations. Original printed page references still refer to the original body of the eBook.

Corrected local validation

The local Python checker now correctly accepts HR Terminated with payroll Stopped. Its earlier mapping incorrectly flagged E1012 and E1041. After correction, the supplied data produces 17 rule findings across 12 employee IDs; all 10 local controls find a defect and release remains BLOCK. The new regression test accepts valid status pairs and detects a contradictory Payable status.

Why the counts differ

A finding row, an affected employee and a lifecycle issue are different units. The historical dashboard starts with 17 lifecycle issues and shows 8 resolved, 2 new and 11 open after the simulated third delivery. Its 19 issue-history rows and 13 affected employees are historical figures. The corrected local checker uses DQ01-DQ07 and RC01-RC03, while the dashboard uses DQ01-DQ06 and RC01-RC04. Equal totals do not establish rule-by-rule parity.

Coverage and remaining implementation limits

The local checker has narrower profile-field reconciliation than the dashboard, a fixed freshness cutoff, and a supplied-data-specific E1042 exception. These are documented limitations, not production-ready generic policies. No fresh Databricks or Azure DevOps deployment is claimed. Consult VALIDATION_SCOPE.md and the machine-readable outputs before interpreting or deploying any component.

Walkthrough and reproducibility

The accompanying full video uses the corrected slides, historical dashboard captures and a disclosed computer-generated narrator. It is a slide walkthrough, not a live screen recording or a voice clone. START_HERE.md explains the review order. PACKAGE_VALIDATION.json records the fresh-extraction tests, and MANIFEST.sha256 covers the packaged files.

Contents

No.	Title	Page
1	Profile: what the three files look like	5
1.1	Rows, keys and duplicates	5
1.2	Blanks, freshness and unexpected patterns	5
1.3	Not every difference is an error	5
1.4	Top three findings and why they matter	6
2	Executable data-quality checks	7
2.1	PySpark: field-level reconciliation	7
2.2	How the checks were verified	7
3	Reconciliation: sources to Employee 360	8
3.1	Summary	8
3.2	Why agreeing totals prove nothing	8
4	Root cause: E1099 appears, E1027 disappears	10
5	Monitoring output and release decision	11
6	Pipeline: Bronze, Silver, Gold, incremental	12
6.1	Layers	12
6.2	Incremental runs and the issue lifecycle	12
7	Running it in Azure Databricks and Azure DevOps	14
8	Proof on Databricks	15
8.1	Bronze, Silver, Gold on Databricks	15
8.2	Dashboard: Employee 360 Data Quality	17
9	AI use	20
App. A	Files, how to run, assumptions, limitations	21
App. B	Evidence index	22

Executive summary

The decision: BLOCK this Employee 360 release

- All 10 rules fail; 5 of them (DQ01, DQ05, RC01, RC02, RC03) block publication.
- Employee 360 is missing a real employee (E1027) and contains an invented one (E1099, a copy of E1028); it shows a terminated employee as active and carries salaries that cannot be traced or verified.

Five things worth knowing

1. Counts and totals lie. 48 rows in every file, yet the key sets differ; the salary gap nets to R64,600 but is R225,200 gross across four employees.

2. The wrong people are listed. E1099 matches E1028 on 8 of 8 attributes and was written by the current build; E1027 vanished. The likely failure point is the identity or crosswalk step (hypothesis).

3. A leaver may be overpaid. E1029 left on 2 October and is still payable at R84,200 a month, while later leavers are stopped. Payroll should confirm before the October cut-off.

4. Some salaries cannot be verified. Payroll holds two salaries for E1015 and a USD amount for E1037. The checks say “cannot verify” instead of guessing.

5. Correct is buildable. The Gold layer rebuilt from the sources by ownership rules has 45 active employees (as HR), the right people, and 40 of 48 fully trusted records.

How to read this book. Chapters 1–5 answer tasks 1–5 of the brief in order; chapters 6–7 answer task 6; chapter 9 answers the AI questions. Each section opens with an “Answers” tag naming the part of the brief it covers. Every figure is labelled with its kind: rendered log (real run output drawn as an image, excerpt kept beside it), chart from data, generated diagram or Databricks run export. Appendix B lists every image with its source and SHA-256.

Requirements index: where each part of the brief is answered

Task	Brief requirement	Answered in	Page
1	Profile with SQL: counts, distinct keys, null rates, duplicates	§1.1, §1.2	5, 5
1	At least one unexpected pattern or freshness concern	§1.2	5
1	Top three findings and their business significance	§1.4, Executive summary	6, 4
2	4–6 automated checks across at least three dimensions	Ch. 2	7
2	Advanced SQL (joins, CTEs, window functions)	Ch. 2	7
2	PySpark for a substantive validation or reconciliation	§2.1	7
2	Return affected employee IDs, not only pass/fail counts	Ch. 2, Ch. 5	7, 11
2	Disclose limitations if Spark cannot run	§2.2, App. A	7, 21
3	Employees missing downstream, unexpected downstream records	§3.1	8
3	Field-level mismatches for business-critical fields	§3.1	8
3	Prove why agreeing totals do not establish correctness	§3.2	8
3	Short reconciliation summary	§3.1	8
4	Evidence, point of failure, impact, confirming information, owner	Ch. 4	10
4	Distinguish proven facts from hypotheses	Ch. 4	10
5	Monitoring output: rule, pass/fail, count, severity, IDs	Ch. 5, §8.2	11, 17
5	An alert threshold; what blocks a release vs is monitored	Ch. 5	11
6	How checks run in Azure Databricks and Azure DevOps CI/CD	Ch. 7, §8.1	14, 15
6	Before deployment, on a schedule, who receives alerts	Ch. 7	14
6	Minimise compute, full-table scans, noisy alerts, paid tools	§6.2, Ch. 7	12, 14
6	One check not to run on every execution, and why	Ch. 7	14
AI	Tools used and what they accelerated	Ch. 9	20
AI	One AI suggestion corrected, simplified, challenged or rejected	Ch. 9	20
AI	How it was verified; was it commercially justified	Ch. 9	20
Submit	SQL, PySpark and scripts in Git; README with run, assumptions, limitations	App. A	21
Submit	Profiling, validation, reconciliation, monitoring outputs, root-cause note	Ch. 1, Ch. 2, Ch. 3, Ch. 4, Ch. 5	5, 7, 8, 10, 11

The 5–10 minute video is recorded separately; its script is docs/walkthrough_script.md.

1. Profile: what the three files look like

Answers Task 1 · inspect with SQL: counts, distinct keys, null rates, duplicates, an unexpected pattern or freshness concern

Three CSV extracts for the October 2026 snapshot: HR (system of record for who works here, status, manager, department, location), payroll (salary, payroll status) and the delivered Employee 360 view under test. The SQL is `sql/01_profile.sql`; it runs unchanged on Databricks and on local Spark.

1.1 Rows, keys and duplicates

Source	Rows	Distinct IDs	Duplicate rows
HR	48	48	0
Payroll	48	47	1
Employee_360	48	48	0

Insight: the counts agree, the contents do not

- All three files have exactly 48 rows. A row-count check, the usual first control, passes.
- Yet payroll has 47 distinct employees (E1015 appears twice with different salaries), and only 47 of Employee 360's IDs exist in HR.

1.2 Blanks, freshness and unexpected patterns

Source	Column	Blank	%
Employee_360	manager_id	5	10.4
HR	department	1	2.1
HR	email	1	2.1
HR	manager_id	5	10.4

Pattern	Rows	Examples
HR dates use yyyy/MM/dd; the other files use ISO yyyy-MM-dd	48	–
HR effective_date after the reporting month (future-dated change)	1	E1042
HR last_updated earlier than effective_date (change keyed in before it takes effect)	23	–
Location alias (CPT for Cape Town)	1	E1040
Payroll currency other than ZAR in a ZAR salary column	1	E1037
Payroll pay_period is not the reporting month	48	–

- **Freshness:** payroll's pay period is 2026-09 for an October snapshot, so salaries are a month behind (acceptable, but it matters for leavers). E1030 has not been touched for 99 days.
- **Format:** HR writes dates as yyyy/MM/dd, the other files as ISO. A loader that assumes ISO turns every HR date into NULL, silently.

1.3 Not every difference is an error

Observation	Verdict	Why
E1040 location “CPT” vs “Cape Town”	Not an error	Same place; compared on the standard name
E1042 HR effective date 19 Nov	Legitimate exception	Future-dated change: the October snapshot should still show the October value

Observation	Verdict	Why
E1001–E1005 have no manager	Allowed	They head departments (they manage others)
23 HR rows updated before their effective date	Normal	Changes keyed in ahead of time
E1029 terminated in HR, payable in payroll	Inconsistent: confirm	September pay may be due, but the two later leavers are already stopped

1.4 Top three findings and why they matter

Answers Task 1 · identify your top three findings and explain their business significance

1 · Employee 360 lists the wrong people and wrong values. Release must be blocked

- E1027 (active, paid R80,300) is missing; E1099 “Unknown Legacy Employee” is present instead and is a field-for-field copy of E1028.
- E1012 is shown Active although HR terminated them; E1018's salary is R6,500 too high; E1037's USD amount is relabelled ZAR; E1044 has a salary with no payroll record.
- Significance: reported headcount is 46 active against HR's 45, and the salary total is off by a net R64,600 that hides R225,200 of gross error.

2 · A leaver is still payable (E1029). Alert payroll before the October cut-off

- HR terminated E1029 on 2026-10-02; payroll still says Payable (R84,200 a month). E1012 and E1041, who left later, are already Stopped.
- E1044 is the mirror case: active in HR with no payroll record at all (possibly unpaid).
- Significance: an overpayment that must later be recovered, or an employee not paid. Both are people and compliance risks, not just data risks.

3 · Payroll, the salary system of record, cannot be trusted for two employees

- E1015 has two payroll rows (R57,100 and R58,600). Employee 360 shows the first, but nobody can say which is right.
- E1037's salary is in USD in a column that should only hold ZAR.
- Significance: until payroll is fixed, salary for these employees is reported as “cannot verify”. Guessing would hide the problem.

2. Executable data-quality checks

Answers Task 2 · 4–6 automated checks across at least three dimensions; advanced SQL; PySpark; return affected employee IDs

Six checks in sql/02_checks.sql, one per dimension. Each is a view that returns one row per affected employee (rule, employee_id, source, detail), so an empty view means pass and a failure always names who is affected.

Rule	Dimension	What it checks	Technique
DQ01	Uniqueness	employee_id unique in every source	window count over a union of sources
DQ02	Completeness	mandatory HR fields; blank manager only for heads	self-join (who manages others), filter(array)
DQ03	Referential integrity	manager_id exists in HR	LEFT ANTI JOIN
DQ04	Consistency	HR status agrees with payroll status	CTE + LEFT JOIN, missing payroll counted
DQ05	Validity	salary is a positive ZAR amount; pay period well formed	RLIKE, CASE
DQ06	Freshness	updated within 30 days of the extract	datediff over a union of sources

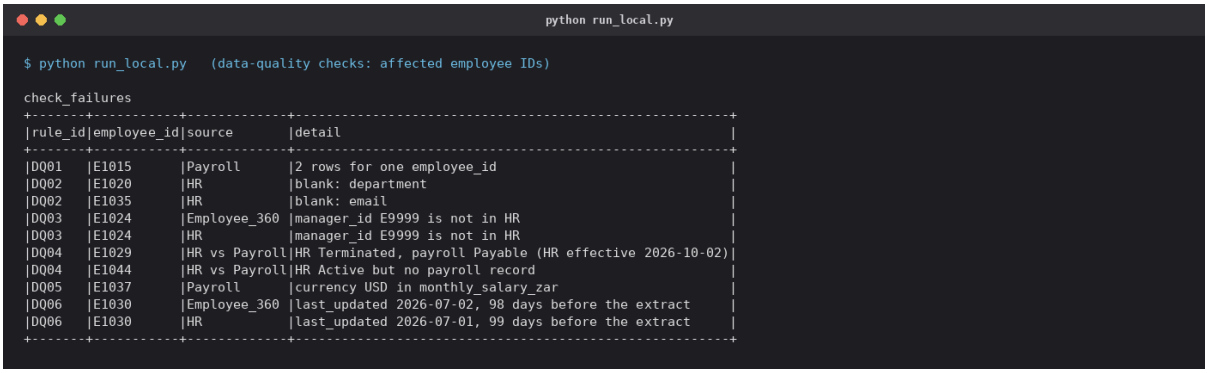


Figure 1. Output of the six checks: every failure names the employee. [Rendered log]

2.1 PySpark: field-level reconciliation

reconcile_fields() in the notebook compares every business-critical field of Employee 360 with the system that owns it (HR or payroll), using null-safe equality, and classifies each difference as an Error, a Legitimate exception or Cannot verify, with the reason. It is the PySpark task the brief asks for, and it feeds rules RC03 and RC04.

2.2 How the checks were verified

tests/check_expected.py recomputes every rule's affected IDs from the raw CSVs in plain Python, with separately written logic and no Spark, and compares them with the Spark output. It runs in seconds, so it can run on every pull request.



Figure 2. Independent recomputation: all 10 rules agree with the Spark output. [Rendered log]

3. Reconciliation: sources to Employee 360

Answers Task 3 · missing downstream, unexpected downstream, field mismatches; why agreeing totals do not prove correctness; a short summary

3.1 Summary

Measure	Value
Employees in HR (system of record)	48
Rows in Employee 360	48
Matched on employee_id	47
Missing downstream (in HR, not in Employee 360)	1
Unexpected downstream (in Employee 360, not in HR)	1
Matched employees with at least one field error	6
Matched employees whose value cannot be verified (conflicting source)	1
Field differences that are legitimate exceptions	1

Employee	Field	Source	Employee 360	Classification
E1015	monthly_salary	57100.00	57100.00	Cannot verify
E1012	employment_status	Terminated	Active	Error
E1018	monthly_salary	62900.00	69400.00	Error
E1037	currency	USD	ZAR	Error
E1044	monthly_salary	blank	56200.00	Error
E1044	currency	blank	ZAR	Error
E1020	department	blank	Finance	Error
E1033	department	People	Sales	Error
E1042	effective_date	2026-11-19	2026-10-01	Legitimate exception

3.2 Why agreeing totals prove nothing

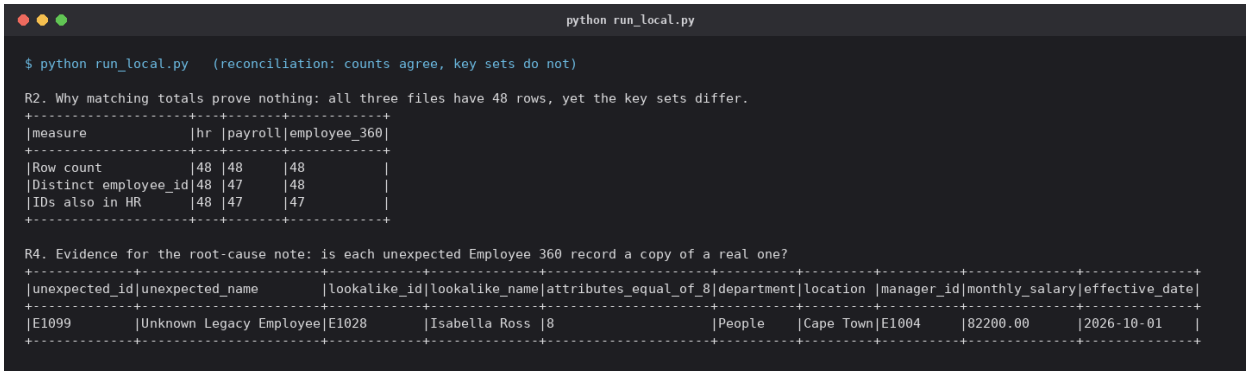


Figure 3. Row counts agree (48/48/48) while the key sets differ; E1099 matches E1028 on 8 of 8 attributes.
[Rendered log]

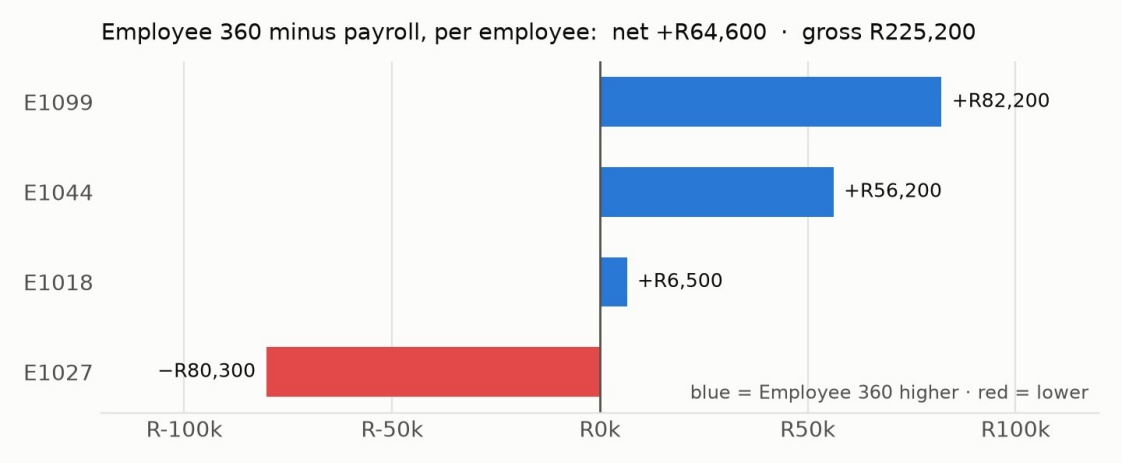


Figure 4. One net figure (R64,600) hides four errors worth R225,200 gross. [Chart from data]

Insight: reconcile keys and fields, not totals

- A missing employee and an invented one cancel exactly in a headcount.
- Salary errors in both directions cancel in a total: +R82,200, −R80,300, +R56,200, +R6,500.
- Only a key-level and field-level comparison against the system of record finds them.

4. Root cause: E1099 appears, E1027 disappears

Answers Task 4 · evidence, likely point of failure, business impact, what would confirm it, who should fix it; facts kept apart from hypotheses

Proven facts (evidence in outputs/)	Hypothesis (not proven)
E1027 is in HR (Active) and payroll (Payable, R80,300) but not in Employee 360 (recon_keys.csv). E1099 exists only in Employee 360. E1099 equals E1028 (Isabella Ross) on 8 of 8 attributes (recon_lookalike.csv). E1099's last_updated (2026-10-08) is the build date stamped on all current rows, so this build wrote it. The 48-row count did not change.	The identity-resolution step that assigns employee_id in the Employee 360 build (for example a legacy-ID crosswalk) produced a second record carrying E1028's attributes under the legacy key E1099, and the same step may be why the neighbouring ID E1027 was dropped. Alternative: a manual legacy insert copied an existing row while a separate filter excluded E1027.

- **Related pattern (hypothesis):** Employee 360 carries values the current extracts do not contain (E1020's department, E1044's salary, E1012's pre-termination status). The build may keep the previous value when the source is blank or missing.
- **What would confirm it:** the crosswalk or MDM entries for E1027, E1028 and E1099; the Employee 360 table's Delta history (DESCRIBE HISTORY, VERSION AS OF) to see when E1099 appeared and E1027 went; the 2026-10-08 job logs and merge logic; HR confirming that no E1099 exists.
- **Business impact:** people reporting lists the wrong people while headcount and cost look plausible; anything fed by Employee 360 (access, org charts) inherits the error.
- **Who should fix it:** the Employee 360 data engineering team (owner of the build and the crosswalk) with the MDM or HR data steward. Release stays blocked by RC01 and RC02 until it is fixed.

5. Monitoring output and release decision

Answers Task 5 · rule, pass/fail, affected count, severity, affected IDs; an alert threshold; what blocks a release and what is monitored

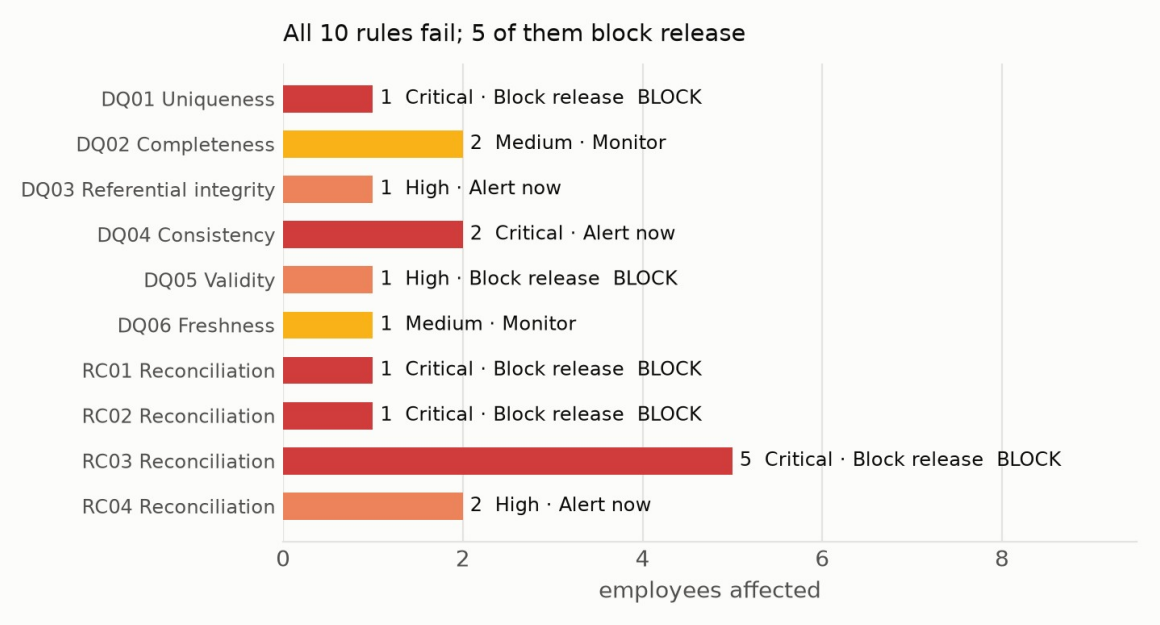


Figure 5. All 10 rules fail; 5 block release. [Chart from data]

Rule	Status	Count	Severity	Action	Alert when >	Affected IDs
DQ01	FAIL	1	Critical	Block release	0	E1015
DQ02	FAIL	2	Medium	Monitor	1	E1020, E1035
DQ03	FAIL	1	High	Alert now	0	E1024
DQ04	FAIL	2	Critical	Alert now	0	E1029, E1044
DQ05	FAIL	1	High	Block release	0	E1037
DQ06	FAIL	1	Medium	Monitor	2	E1030
RC01	FAIL	1	Critical	Block release	0	E1027
RC02	FAIL	1	Critical	Block release	0	E1099
RC03	FAIL	5	Critical	Block release	0	E1012, E1015, E1018, E1037, E1044
RC04	FAIL	2	High	Alert now	0	E1020, E1033

- **Block release** (any failure stops this Employee 360 build being published): wrong people, wrong or unverifiable pay, invalid currency, duplicate keys.
- **Alert now** (owner told the same day; publishing may continue): the fix is in a source system. DQ04, a leaver still payable, goes to payroll before the pay-run cut-off.
- **Monitor** (trend, alert only above a threshold): e.g. alert when more than 1 employee (2% of 48) has a blank mandatory field; freshness alerts when more than 2 records are stale.

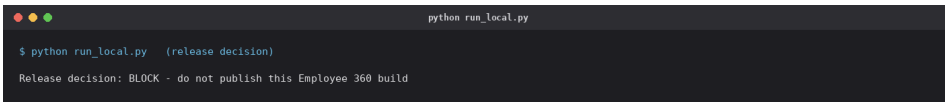


Figure 6. The gate's decision for this delivery. [Rendered log]

6. Pipeline: Bronze, Silver, Gold, incremental

Answers Task 6 · minimise repeated full-table scans and noisy alerts; how the checks sit in a Databricks pipeline

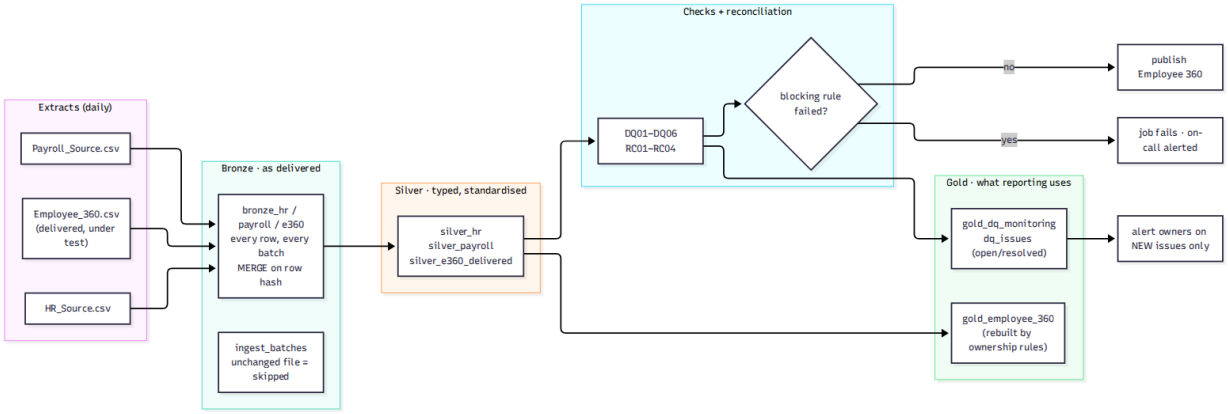


Figure 7. How the checks sit in the pipeline: a blocking failure stops publication; owners hear about new issues only. [Generated diagram]

6.1 Layers

Layer	Table	Rows
Bronze	bronze_hr	48
Bronze	bronze_payroll	48
Bronze	bronze_e360	48
Silver	silver_hr	48
Silver	silver_payroll	48
Silver	silver_e360_delivered	48
Gold	gold_employee_360	48
Gold	gold_dq_monitoring	10
Gold	dq_issues	17

Measure	Gold	Delivered Employee 360
Employees	48	48
Active employees	45	46
Employees with a verified ZAR salary	45	48
Trusted records (no open source-quality rule, salary verified)	40	–

Insight: Gold shows what correct looks like

- Gold is rebuilt from Silver by the ownership rules: HR decides who exists and their status; payroll supplies salary only when it holds exactly one valid ZAR value.
- It has E1027 and not E1099, shows E1012 as Terminated and gives E1018's salary as R62,900. 45 active employees, the same as HR.
- Salaries for E1015, E1037 and E1044 are left empty and flagged, not guessed. 40 of 48 records are fully trusted.

6.2 Incremental runs and the issue lifecycle

Each delivery is MERGEd into Bronze on a row hash. An extract whose file hash matches the last one is not read at all. Failures are MERGEd into dq_issues (open, resolved), and alerts go

out only for issues that are new in a run. Run 3 uses a labelled, simulated delivery (data/simulated_run3) with three fixes and one new problem.

Run	Delivery	Employees changed	Checks	New (alerted)	Resolved	Open
1	data	49	ran	17	0	17
2	data	0	skipped	0	0	17
3	data/simulated_run3	7	ran	2	8	11

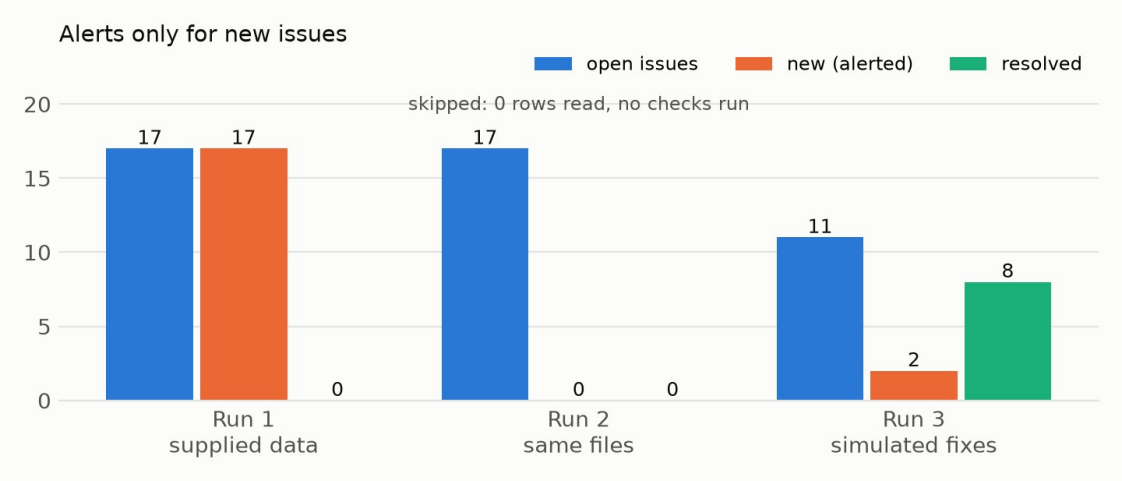


Figure 8. Run 2 reads nothing; run 3 alerts on 2 new issues, not on the 11 still open. [Chart from data]

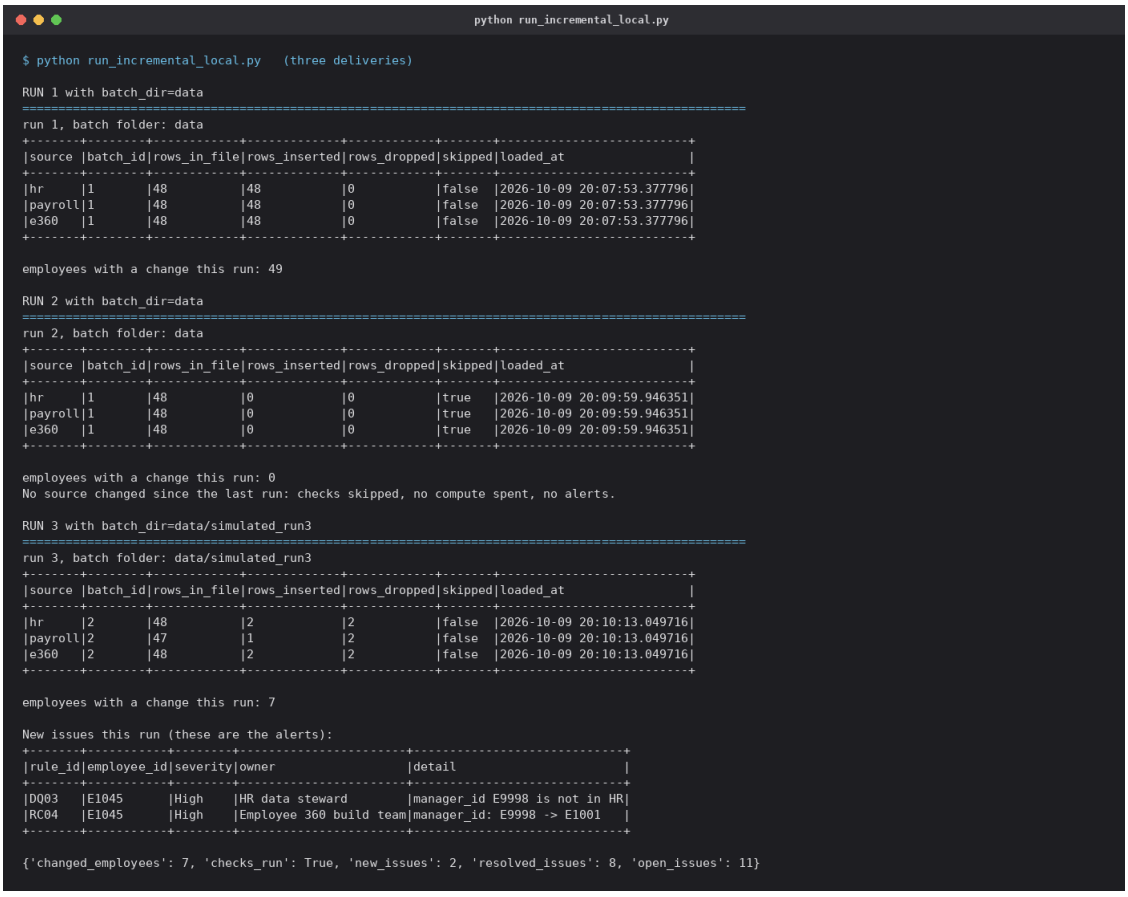


Figure 9. The three runs: batches loaded or skipped, changed employees, and the only two alerts. [Rendered log]

7. Running it in Azure Databricks and Azure DevOps

Answers Task 6 · what runs before deployment, on a schedule, who is alerted; compute cost, scans, noisy alerts, paid tools; one check not run every time

When	What runs	Databricks cost
Every pull request	run_local.py on the fixture CSVs and tests/check_expected.py (must be 10/10), on the build agent with local PySpark	none
Merge to main	databricks bundle validate and deploy -t test, then one smoke run on the fixtures	one short run
Release	databricks bundle deploy -t prod behind an approval gate	none
Daily 06:00 (prod)	The medallion job: Bronze load, checks with fail_on_block=true, Silver and Gold; publish runs only if the checks task succeeds	minutes, serverless

- **Alerts:** DQ04 (leaver still payable) to payroll and HR operations the same day, re-checked three working days before the pay run; a blocking failure to the Employee 360 on-call through the job's failure notification; High rules to the HR data steward for new issues only; Medium rules in a weekly digest.
- **Compute:** serverless jobs compute (nothing billed between runs); unchanged extracts skipped by file hash; only new or changed rows written; at scale the row-level checks filter to the changed keys the notebook already lists.
- **Noise:** one issue per rule and employee; alert once when it opens, close it when it stops failing.
- **Tools:** plain SQL and PySpark in the same job, history in Delta tables; no separate data-quality or observability licence. The same SQL runs on open-source Spark, so CI needs no workspace.

A check I would not run on every execution

- The full profile (every column of every table) and the look-alike search (a self-join that grows with the square of the table).
- They give diagnostic insight, not a pass/fail gate, and their answers change slowly: run them weekly and during an investigation.

8. Proof on Databricks

Answers Tasks 2 and 6 · the same notebooks on Databricks serverless, results compared with the local run

The notebooks and SQL files were imported into the workspace unchanged and run as one-time serverless jobs. Results were read back with the SQL Statement API and compared line by line with the local outputs (evidence/databricks_run.md, raw data in evidence/databricks_*.csv).

#	Notebook	Run ID	Result	Duration
0	employee360_dq	898819557289071	FAILED	126 s
1	employee360_dq	515886583443770	SUCCESS	136 s
2	employee360_incremental	981987682878372	SUCCESS	225 s
3	employee360_incremental	94742837183686	SUCCESS	37 s
4	employee360_incremental	68446115877254	SUCCESS	196 s

Comparison with the local run	Result
Monitoring: rules with identical status and affected IDs	10 of 10
Incremental runs: changed / checks / new / resolved / open	3 of 3 runs identical
Issue lifecycle rows (status, first/last seen, resolved, detail)	19 of 19 identical

Insight: one engine is not enough evidence

- The first Databricks run failed: a correlated scalar subquery that open-source Spark accepts is rejected by Databricks serverless (MUST_AGGREGATE_CORRELATED_SCALAR_SUBQUERY).
- The fix carries the value through the existing join (same result) and both engines now pass. CI runs the open-source engine, so the deploy smoke run on Databricks stays in the pipeline for exactly this reason.

8.1 Bronze, Silver, Gold on Databricks

Three runs of the medallion notebook, each named for its purpose so the Runs list explains itself. All succeeded on the first attempt with no code change; Gold after run 1 matches the local output in all 576 cells, and the run log, all 19 issue rows and all 10 monitoring rules match too.

Run name	Result	Duration
Medallion run 1 of 3 | supplied data, full load	SUCCESS	780 s
Medallion run 2 of 3 | same files again, should skip	SUCCESS	34 s
Medallion run 3 of 3 | simulated fixes + 1 new issue	SUCCESS	293 s



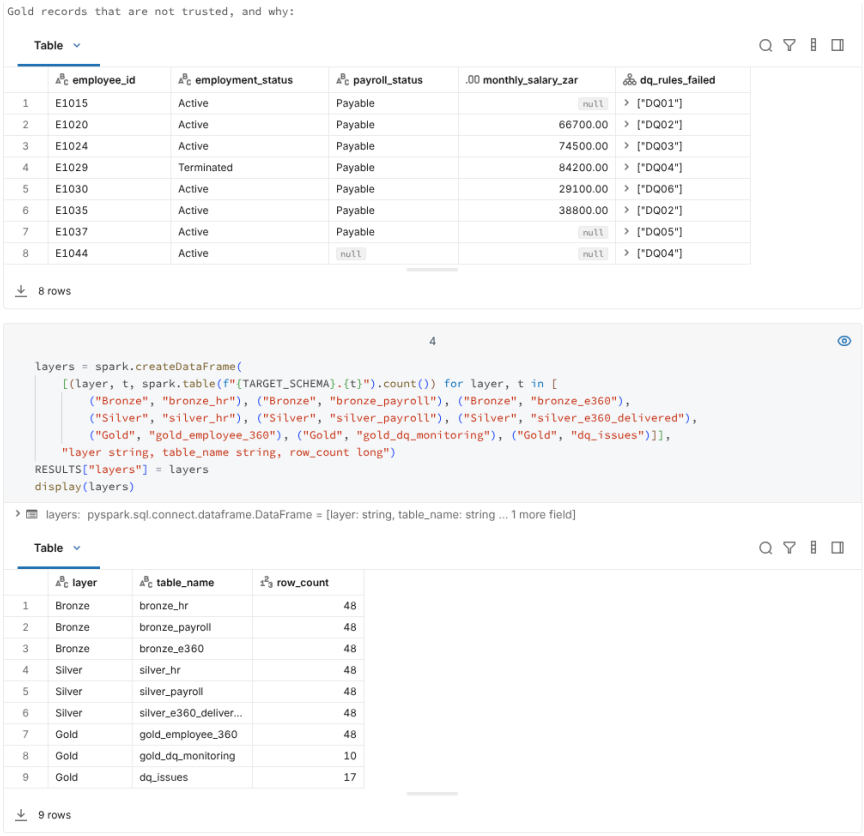
Table

	rule_id	rule	dimension	status	affected_count
1	DQ01	employee_id unique in every source	Uniqueness	FAIL	
2	DQ02	Mandatory HR fields populated	Completeness	FAIL	
3	DQ03	manager_id exists in HR	Referential integrity	FAIL	
4	DQ04	HR status agrees with payroll status	Consistency	FAIL	
5	DQ05	Payroll salary is a positive ZAR amount	Validity	FAIL	
6	DQ06	Record updated within 30 days of the extract	Freshness	FAIL	
7	RC01	Every HR employee is in Employee 360	Reconciliation	FAIL	
8	RC02	No Employee 360 record without an HR employee	Reconciliation	FAIL	
9	RC03	Status, salary and currency match (and can be verified against) their source	Reconciliation	FAIL	
10	RC04	Department, manager, location and name match HR	Reconciliation	FAIL	

10 rows | 29.631s runtime

Release decision: BLOCK - do not publish this Employee 360 build

Figure 10. Run 1 on Databricks: the monitoring table (10 rules, all FAIL) and the release decision BLOCK.
[Databricks run export (rendered, not a browser screenshot)]



Gold records that are not trusted, and why:

Table

	employee_id	employment_status	payroll_status	monthly_salary_zar	dq_rules_failed
1	E1015	Active	Payable	null	["DQ01"]
2	E1020	Active	Payable	66700.00	["DQ02"]
3	E1024	Active	Payable	74500.00	["DQ03"]
4	E1029	Terminated	Payable	84200.00	["DQ04"]
5	E1030	Active	Payable	29100.00	["DQ06"]
6	E1035	Active	Payable	38800.00	["DQ02"]
7	E1037	Active	Payable	null	["DQ05"]
8	E1044	Active	null	null	["DQ04"]

8 rows

4

```
layers = spark.createDataFrame(
    [(layer, t, spark.table(f"{TARGET_SCHEMA}.{t}").count()) for layer, t in [
        ("Bronze", "bronze_hr"), ("Bronze", "bronze_payroll"), ("Bronze", "bronze_e360"),
        ("Silver", "silver_hr"), ("Silver", "silver_payroll"), ("Silver", "silver_e360_delivered"),
        ("Gold", "gold_employee_360"), ("Gold", "gold_dq_monitoring"), ("Gold", "dq_issues")]],
    "layer string, table_name string, row_count long")
RESULTS["layers"] = layers
display(layers)
```

layers: pyspark.sql.connect.dataframe.DataFrame = [layer: string, table_name: string ... 1 more field]

Table

	layer	table_name	row_count
1	Bronze	bronze_hr	48
2	Bronze	bronze_payroll	48
3	Bronze	bronze_e360	48
4	Silver	silver_hr	48
5	Silver	silver_payroll	48
6	Silver	silver_e360_deliver...	48
7	Gold	gold_employee_360	48
8	Gold	gold_dq_monitoring	10
9	Gold	dq_issues	17

9 rows

Figure 11. Run 1 on Databricks: Gold records that are not trusted, and why; row counts per layer. [Databricks run export (rendered, not a browser screenshot)]



run 2, batch folder: data

log_df: pyspark.sql.connect.dataframe.DataFrame = [source: string, batch_id: integer ... 6 more fields]

Table

	source	batch_id	rows_in_file	rows_inserted	rows_dropped	skipped	loaded_at
1	hr	1	48	0	0	true	2026-10-09T21:1
2	payroll	1	48	0	0	true	2026-10-09T21:1
3	e360	1	48	0	0	true	2026-10-09T21:1

3 rows | 6.913s runtime

bronze: pyspark.sql.connect.dataframe.DataFrame = [employee_id: string, full_name: string ... 13 more fields]

changed_keys: pyspark.sql.connect.dataframe.DataFrame = [employee_id: string]

```
employees with a change this run: 0
```

```
skipped: no changes
```

Command skipped

Command skipped

Command skipped

Figure 12. Run 2 on Databricks: same files again, 0 rows inserted, checks skipped. [Databricks run export (rendered, not a browser screenshot)]



New issues this run (these are the alerts):

Table

	rule_id	employee_id	severity	owner	detail
1	DQ03	E1045	High	HR data steward	manager_id E9998 is not in ...
2	RC04	E1045	High	Employee 360 build te...	manager_id: E9998 → E1001

2 rows | 28.531s runtime

```
{'changed_employees': 7, 'checks_run': True, 'new_issues': 2, 'resolved_issues': 8, 'open_issues': 11}
```

Figure 13. Run 3 on Databricks (simulated delivery): the only two alerts, and the run summary. [Databricks run export (rendered, not a browser screenshot)]

8.2 Dashboard: Employee 360 Data Quality

Answers Task 5 · a usable monitoring output, live in Databricks AI/BI

A Databricks AI/BI dashboard on the Gold and monitoring tables, deployed from dashboards/employee360_dq.lvdash.json. Headline numbers come from the supplied data (run 1, kept with Delta time travel); anything from run 3 is labelled as the simulated delivery.

Page	What it shows
1 · Release gate (supplied data)	Decision BLOCK, 10 of 10 rules failing, 5 blocking, affected employees by rule and severity, the full rule table with owners
2 · Reconciliation and Gold	Reconciliation categories (below), Gold against the delivered Employee 360, untrusted Gold records, failure detail by rule
3 · Pipeline runs and issue lifecycle	New, resolved and open issues per run; the issue table by status; the load log per source

Reconciliation category	Employees	Counts as failure	Meaning
Missing downstream	E1027	yes	In HR and payroll but missing from Employee 360 (RC01)
Unexpected downstream	E1099	yes	In Employee 360 with no HR or payroll record (RC02)
Field error	E1012, E1018, E1020, E1033, E1037, E1044	yes	Employee 360 differs from, or cannot be traced to, the system of record (RC03, RC04)
Cannot verify	E1015	yes	The system of record is ambiguous, so the Employee 360 value cannot be verified (RC03)
Legitimate exception	E1042	no	A difference that is correct (e.g. a future-dated HR change); shown for transparency, does not count as a failure or block release

Access: private to the owner

- Owner-only (plus the workspace admins group, which on this workspace is the same person); no shares, no public or embed links, no schedules or subscriptions.
- Published with embedded credentials off, so anyone ever given access would still need their own grant on the tables. Evidence: evidence/databricks_dashboard.md.

Screenshots taken by the candidate in their own signed-in browser, cropped only to remove the browser bar and taskbar (evidence/dashboard_screenshots.md has the crop boxes and the SHA-256 of each original).

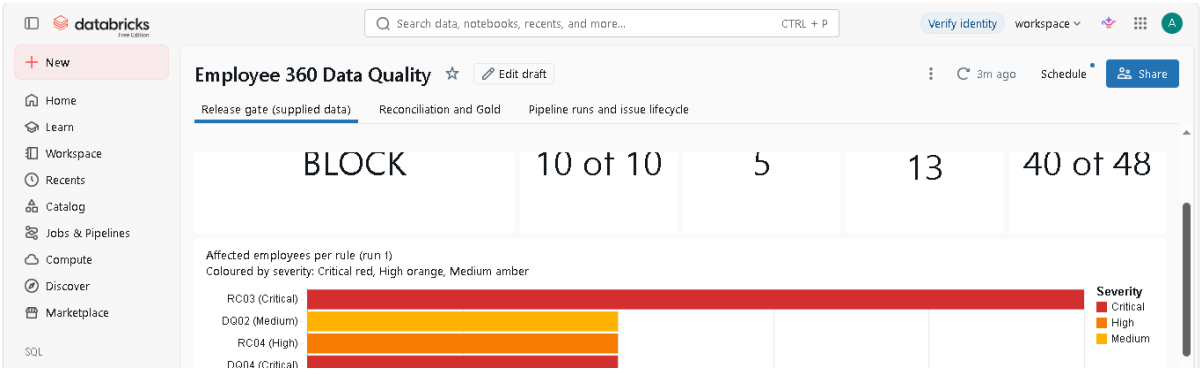


Figure 14. Dashboard page 1, Release gate: BLOCK, 10 of 10 rules failing, 5 blocking, 13 employees affected, 40 of 48 Gold records trusted; affected employees per rule by severity (cropped above a desktop notification that covered the lower chart). [Screen capture (candidate's browser)]

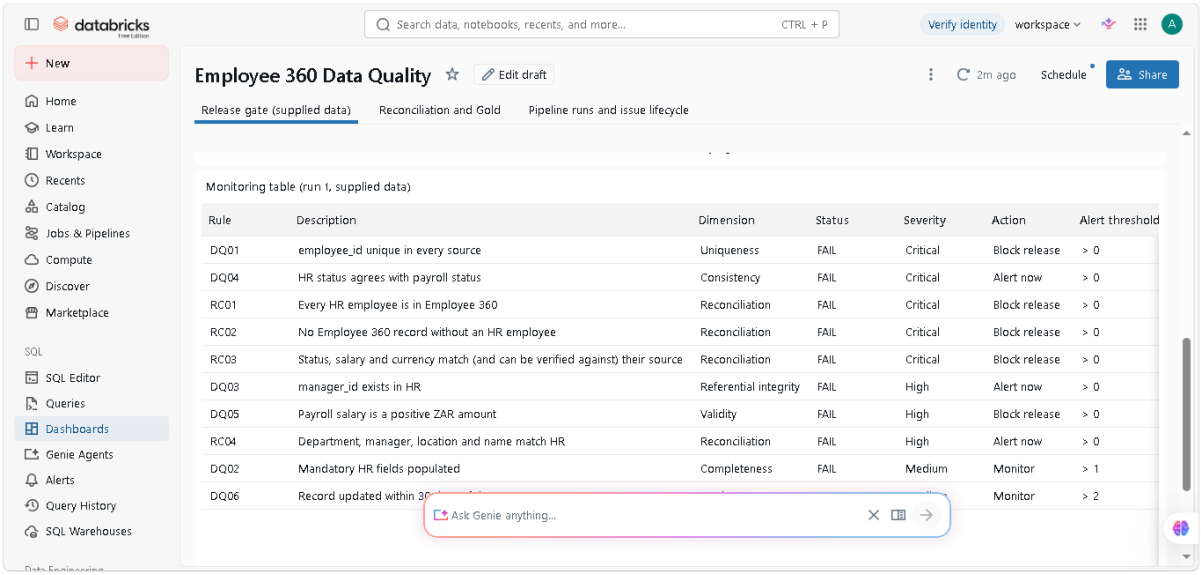


Figure 15. Dashboard page 1: the monitoring table for run 1 (supplied data), with severity, action and alert threshold. [Screen capture (candidate's browser)]

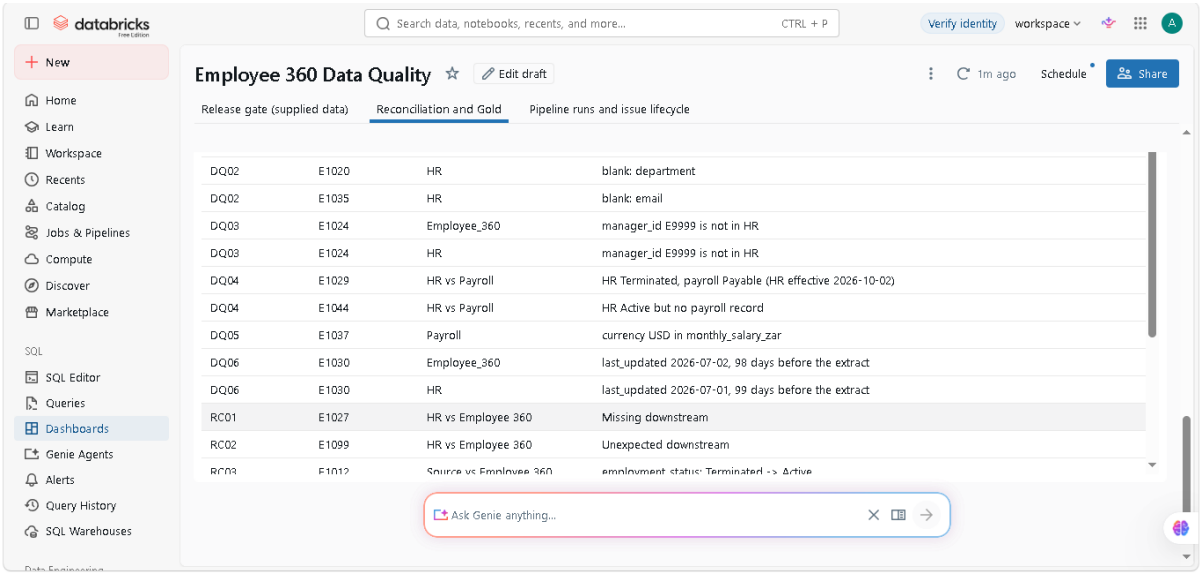


Figure 16. Dashboard page 2: failure detail by rule and employee. [Screen capture (candidate's browser)]

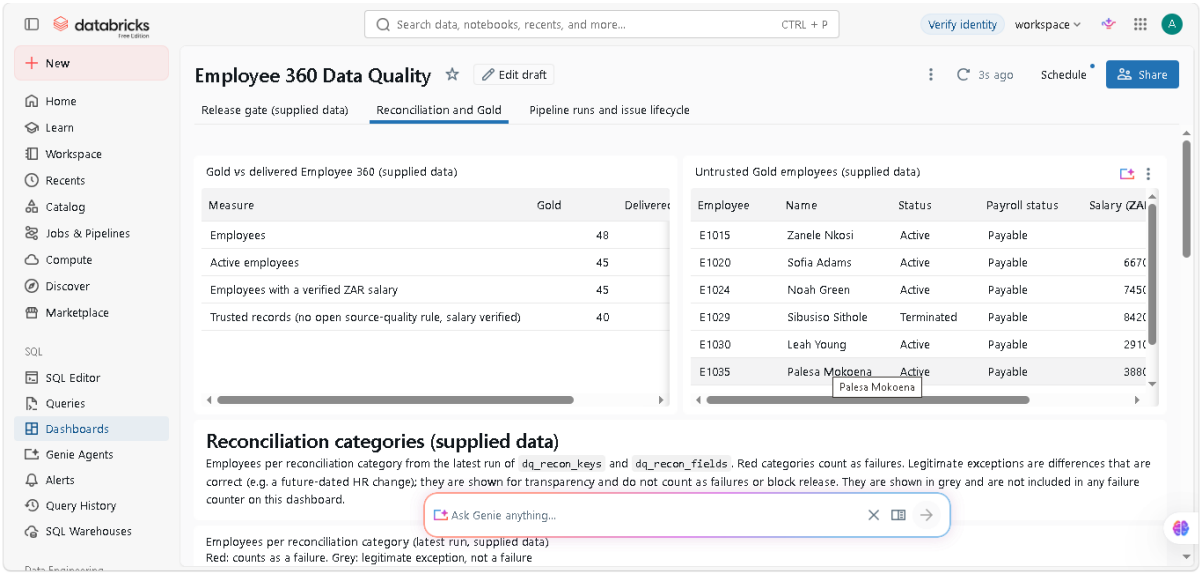


Figure 17. Dashboard page 2: Gold against the delivered Employee 360 (45 vs 46 active), and the untrusted Gold employees. [Screen capture (candidate's browser)]

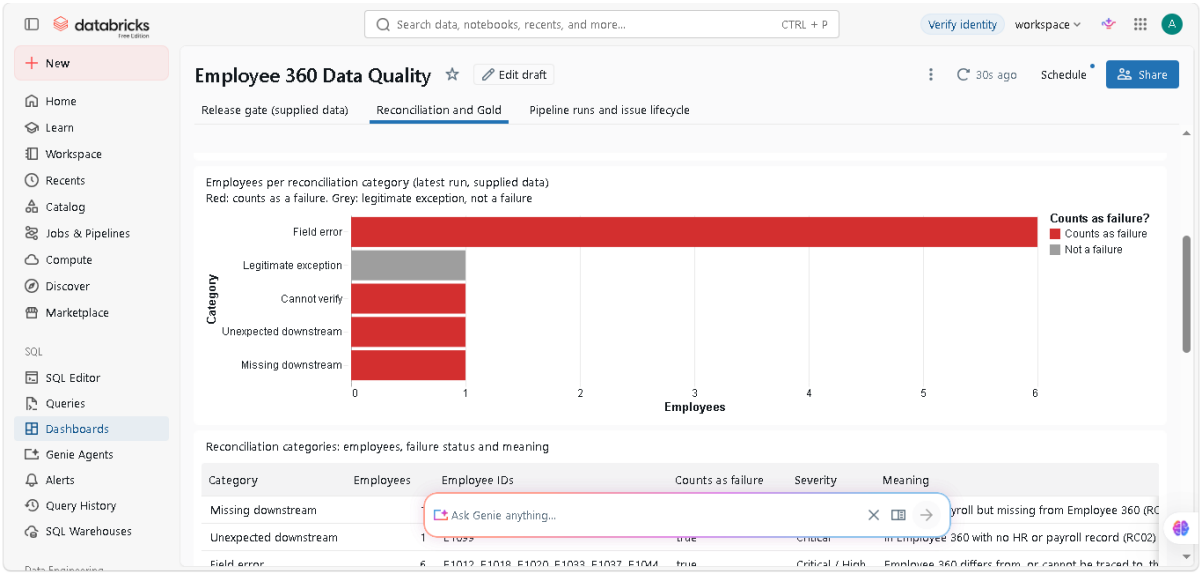


Figure 18. Dashboard page 2: employees per reconciliation category. The legitimate exception is grey and not counted as a failure. [Screen capture (candidate's browser)]

9. AI use

Answers AI use · tools and what they sped up; one suggestion corrected or rejected; how it was verified; commercial value

- **Tools:** Claude Code (Anthropic) drafted and ran the SQL, PySpark, runners and these notes; Codex (OpenAI) reviewed the branch independently. Judgement calls (what is an error, severity, ownership) were not delegated.
- **Corrected:** the first PySpark reconciliation used groupBy().agg(F.first()), which is non-deterministic in Spark and let E1015 pass silently because Employee 360 matched one of two conflicting salaries. The independent profile caught it; the fix takes values in file order and adds a Cannot verify class.
- **Also corrected:** a draft claim that the salary gap was under 0.3% (it is 2.6%, and the real story is net against gross); a QUALIFY clause that runs on Databricks but not on open-source Spark, rewritten so one SQL file runs in both.
- **Verification:** the independent plain-Python recomputation (10/10), the same code on Databricks and locally, and every finding traceable to an output file and employee ID.
- **Commercial value:** worth it for mechanical, checkable work (scaffolding, harness, drafts); no time saved on judgement, and some spent on verification, which the silent-pass example shows is necessary. With real client data, only a client-approved assistant inside the client's tenancy, or schemas and synthetic samples only.

Appendix A. Files, how to run, assumptions, limitations

Answers What to submit · SQL, PySpark and scripts in Git; outputs; README with run instructions, assumptions and limitations

Path	What it is
sql/00_staging.sql ... 03_reconciliation.sql	Typed staging, profile, six checks, key and totals reconciliation
sql/10_silver_gold.sql	Silver and Gold layers
notebooks/employee360_dq.py	Databricks notebook: runs the SQL, PySpark reconcile_fields, monitoring, release gate
notebooks/employee360_incremental.py, employee360_medallion.py	Incremental Bronze load, issue lifecycle, Silver and Gold
run_local.py, run_incremental_local.py	Run the notebooks on local PySpark (+ Delta) and save outputs/
tests/check_expected.py	Independent plain-Python recomputation of every rule
outputs/*.csv, outputs/logs/	Every result in this book, and the run logs
databricks.yml, azure-pipelines.yml	Bundle job and CI/CD pipeline (not executed: no Azure project available)
docs/, README.md	Notes, assumptions, limitations, video script

Run locally: `pip install "pyspark==4.0.*" "delta-spark==4.0.*"; python run_local.py; python tests/check_expected.py; python run_incremental_local.py`

- **Assumptions:** employee_id is the join key; HR owns status, manager, department, location; payroll owns salary, currency and payroll status; extract date 2026-10-08; stale = not updated in 30 days; future-dated = HR effective date after 31 October; CPT = Cape Town; payroll period 2026-09 is the latest for the October snapshot.
- **Limitations:** databricks.yml and azure-pipelines.yml were not executed; run 3 of the incremental demo uses a labelled simulated delivery; the root cause is a hypothesis until the crosswalk, Delta history and job logs are seen; at this size the row-level checks re-run on the full snapshot when anything changes.

Appendix B. Evidence index

Every image in this book, what kind it is and where it came from. Check any file with sha256sum <path>.

Fig.	File	Kind	Source	SHA-256 (first 16)
1	term_checks.png	Rendered log	outputs/logs/run_local.log → ebook/captures/term_checks.txt	66c3de0e6277c27f
2	term_independent.png	Rendered log	outputs/logs/check_expected.log → ebook/captures/term_independent.txt	04f926781b7db1b9
3	term_counts.png	Rendered log	outputs/logs/run_local.log → ebook/captures/term_counts.txt	6d2a8dfb6ad1f616
4	chart_net_vs_gross.png	Chart from data	outputs/recon_salary_totals.csv	d0cb99d3f7de9695
5	chart_monitoring.png	Chart from data	outputs/monitoring.csv	5db3620a8b4b6ddc
6	term_release.png	Rendered log	outputs/logs/run_local.log → ebook/captures/term_release.txt	55f8010f10c23b0b
7	diagram_pipeline.png	Generated diagram	ebook/pipeline.mmd	74f2a7c92b02c2bf
8	chart_issue_lifecycle.png	Chart from data	outputs/incremental_runs.csv	c07c3b8bc1084df0
9	term_incremental.png	Rendered log	outputs/logs/run_incremental_local.log → ebook/captures/term_incremental.txt	6d9d54ac51da5bf6
10	databricks_run1_monitoring.png	Databricks run export (rendered, not a browser screenshot)	evidence/databricks_exports/	2d221abca7ae49fb
11	databricks_run1_gold.png	Databricks run export (rendered, not a browser screenshot)	evidence/databricks_exports/	3732076c0519fe07
12	databricks_run2_skipped.png	Databricks run export (rendered, not a browser screenshot)	evidence/databricks_exports/	ff8036048942d98c
13	databricks_run3_new_issues.png	Databricks run export (rendered, not a browser screenshot)	evidence/databricks_exports/	f30e927f352ca320
14	genuine_dashboard_1_counters_chart.png	Screen capture (candidate's browser)	evidence/dashboard_screenshots.md	a1a9709d7f57f7bd
15	genuine_dashboard_1_rule_table.png	Screen capture (candidate's browser)	evidence/dashboard_screenshots.md	de3bf6a4978fe1d4
16	genuine_dashboard_2_failure_detail.png	Screen capture (candidate's browser)	evidence/dashboard_screenshots.md	ab22abc5739663d4
17	genuine_dashboard_2_gold.png	Screen capture (candidate's browser)	evidence/dashboard_screenshots.md	6343696869d9ec59
18	genuine_dashboard_2_recon_categories.png	Screen capture (candidate's browser)	evidence/dashboard_screenshots.md	f9ca9f27b6873532

Appendix C | Logical model and join map

Clear join keys and grains

1. Source tables

HR_SOURCE

PK employee_id

full_name / department

manager_id / status

effective_date

PAYROLL_SOURCE

PK employee_id

PK pay_period

salary / currency

payroll_status

EMPLOYEE_360

PK employee_id

profile attributes

salary / currency

status

2. Reconciliation joins

From	To	Join predicate	Reason
HR_SOURCE	EMPLOYEE_360	h.employee_id = e.employee_id	Missing / unexpected records
PAYROLL_SOURCE	EMPLOYEE_360	p.employee_id = e.employee_id	Salary, currency, payroll status
PAYROLL_SOURCE	current payroll view	employee_id + pay_period	Preserves payroll grain
HR_SOURCE	HR_SOURCE	manager_id = employee_id	Manager integrity control

3. Curated pipeline

BRONZE_INGEST_BATCH

PK batch_id

FK run_id

source_name + file_hash

rows / inserted / skipped

SILVER_EMPLOYEE

PK employee_id

FK batch_id

conformed attributes

verification_status

GOLD_EMPLOYEE_360

PK employee_id

trusted attributes

verified_salary

trust_status

4. Operational evidence relationships

Relationship	Join keys	Meaning
PIPELINE_RUN -> BRONZE	run_id	A run creates one or more ingested batches
PIPELINE_RUN -> DQ_MONITORING	run_id + rule_id	Rule outcome and affected count per run
DQ_MONITORING -> DQ_ISSUE	run_id + rule_id	Rule outcome drills to affected employees

Key design point: payroll is not joined on employee_id alone when analysing historical periods. Its natural grain is employee_id + pay_period

Appendix D | Kimball dimensional design

Applied to Employee 360

Purpose: turn the Employee 360 analytical layer into a clear, governed dimensional model. This is an applied design note; it does not change the supplied-data findings or the release decision.

1. Start with the business process and state the grain

The analytics process is Employee 360 reconciliation. The primary Gold record has grain: one employee version for one effective period, produced by one pipeline run. Declaring this first prevents mixed-level joins, duplicate counts and misleading salary totals.

2. Use conformed employee and date dimensions

DimEmployee carries the durable employee business key plus descriptive attributes needed to analyse status, department, manager and location. DimDate is role-playing: it supports effective date, source extract date, load date and issue detected date. The same definitions must be reused by the HR, payroll and Employee 360 processes so dashboard comparisons mean the same thing.

3. Separate descriptive context from measurements

Employee attributes belong in dimensions. Numeric measures such as salary amount, affected-record count, rows loaded and issues opened/resolved belong in facts. Salary retains currency alongside amount; it is not summed across currencies until a defined conversion rule exists.

4. Preserve history deliberately

Use a Type 2 slowly changing DimEmployee for analytically important changes such as department, manager, location and employment status. Each version has effective-from, effective-to and current-row fields. This explains why a record was considered valid at a point in time, including the future-dated E1042 legitimate exception.

5. Make auditability a first-class analytical process

PipelineRun, DQMonitoring, DQIssueLifecycle and ReconciliationDetail are audit / error-event facts at their own declared grains. They retain run ID, source/batch, rule, severity, affected employee and status so an operator can trace a dashboard signal back to a specific input and run.

Design result: a compact Employee 360 star-style presentation layer with shared dimensions and separate operational facts. It supports trustworthy employee analysis without hiding reconciliation failures.

Appendix E | How to read the three runs

The run sequence tests pipeline behaviour. It is evidence about how the solution responds to deliveries; it is not evidence that the original supplied files were corrected.

Run 1 - supplied data, full load

The supplied HR, payroll and Employee 360 files load and the checks identify the real data-quality issues. The release gate is BLOCK because critical source and reconciliation controls fail.

Run 2 - identical delivery, correctly skipped

The same files are delivered again. Their hashes are unchanged, so the incremental-load logic skips them. No duplicate rows should be created. Existing issues remain open because no corrected source data arrived.

Run 3 - simulated delivery, controlled test

A separate test delivery is constructed to exercise change handling. It resolves 8 previously open issues, introduces 2 new test issues and leaves 11 open. The dashboard labels it SIMULATED so no reader can mistake this for a correction to the supplied files.

Open versus resolved

Open means the latest assessed delivery still contains the problem and requires action. Resolved means a later delivery no longer triggers the same rule for that employee. In this challenge, “resolved” on Run 3 means resolved in the simulated test delivery only; it does not rewrite the original supplied evidence.

Technical failure versus data-quality failure

A technical failure means the job could not run (for example, the earlier unsupported SQL correlated-subquery error). A data-quality failure means the job ran successfully and found a source or reconciliation defect. They are investigated differently: engineers restore a failed run; data owners correct a data defect.

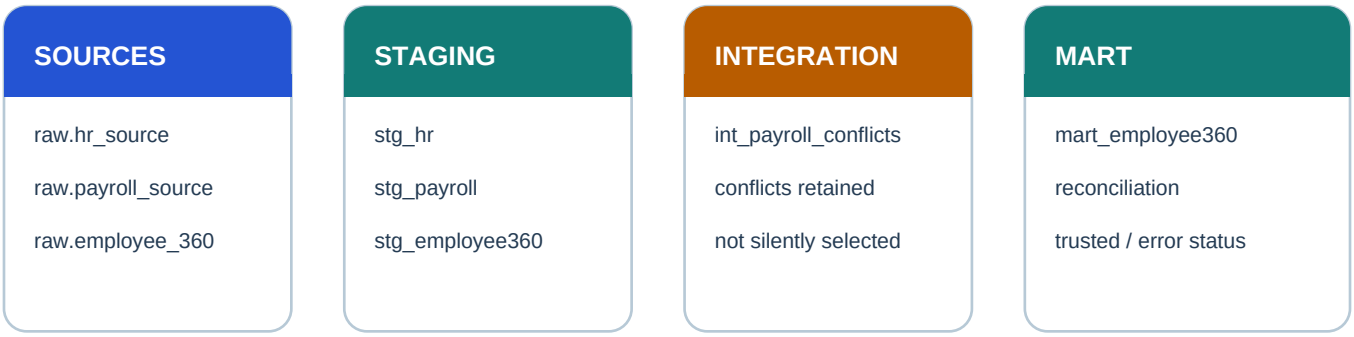
Reference: Kimball, Ralph and Margy Ross. The Data Warehouse Toolkit, 3rd ed., Wiley, 2013. Applied concepts: four-step design process, declared grai dimensions and facts, conformed dimensions, Type 2 history, audit dimensions and error-event schemas.

Appendix F | dbt governed SQL layer

Added to the submission package

Why dbt adds value here

dbt makes the SQL transformation layer versioned, documented, testable and visible as lineage. It complements the PySpark ingestion an



What the dbt project contains

Item	Purpose	How it strengthens the assignment
Source definitions	Declares HR, payroll and Employee 360 inputs	Explicit lineage from raw tables
Staging models	Standardises types and blank handling	Readable, reusable SQL models
Conflict model	Keeps conflicting payroll values visible	Prevents arbitrary first-value logic
Reconciliation mart	Builds current-period comparison	Clear Gold-layer analytical output
Schema tests	not_null and unique business-key tests	Automated regression checks in CI

Controlled delivery

1. PySpark ingests files, hashes deliveries, runs controls and writes Delta evidence.
2. dbt builds documented SQL models and runs model tests in the Databricks test environment.
3. CI blocks promotion if the agreed tests fail; source-data defects remain visible as data-quality findings.

Verified locally

The included dbt project parsed successfully and executed all five models against a local DuckDB copy of the supplied data.

For the Databricks target, configure the supplied profiles.yml.example with the workspace SQL warehouse connection.